

UNIT III: GRAMMARS, AMBIGUITY, RLG-TO-LLG & DERIVATIONS

Continuous Concurrent Evaluation (CCE2) — Comprehensive Worked Solutions

Course Pattern: TE IT - 2024 Pattern (SPPU)
Document Type: CCE2 Model Answer Solutions

Academic Term: Semester V (2026)
Course Code: PCC-303-ITT

NOTE:

This document provides exhaustive, step-by-step worked solutions for all questions in **Unit III (Context-Free Grammars & Regular Grammars)** from the official CCE2 question bank. It covers: (1) Ambiguity proof in calculator expression parser using dual LMDT/RMDT, (2) Right-Linear to Left-Linear Grammar (RLG-to-LLG) conversions via Finite Automata reversal, and (3) Complete Leftmost and Rightmost Derivations across all real-world case study systems.

Question 1: Ambiguity in Calculator Expression Parser [8 Marks]

SCENARIO:

A software engineering team is developing a basic calculator application. The calculator accepts expressions containing the operand a and the operators $+$ and $*$. The expression grammar used by the parser is:

$$X \rightarrow X + X \mid X * X \mid a$$

Question: Apply the concept of ambiguity in Context-Free Grammars to the calculator parser. Check whether the grammar is ambiguous for the string $w = a + a * a$. Demonstrate your answer by constructing two distinct Leftmost Derivations (LMDT) and two distinct Parse Trees, and conclude whether the grammar is ambiguous. [8 Marks]

1. Theoretical Definition of Ambiguity [1 Mark]

A Context-Free Grammar (CFG) G is called **ambiguous** if there exists at least one terminal string $w \in L(G)$ for which there can be constructed:

1. Two or more distinct **Leftmost Derivations (LMDT)**, OR
2. Two or more distinct **Rightmost Derivations (RMDT)**, OR
3. Two or more distinct **Parse Trees (Derivation Trees)**.

Ambiguity is undesirable in programming language compilers and parsers because it leads to non-deterministic semantic interpretation (e.g., conflicting operator precedence).

2. Construction of Two Distinct Leftmost Derivations (LMDT) [3 Marks]

We derive the target string $w = a + a * a$ from start symbol X using two completely different leftmost derivation sequences:

Leftmost Derivation 1 (LMDT-1):
(Multiplication parsed at root: $X * X$)

$$\begin{aligned} X &\Rightarrow_{\text{lm}} X * X \\ &\Rightarrow_{\text{lm}} (X + X) * X \\ &\Rightarrow_{\text{lm}} (a + X) * X \\ &\Rightarrow_{\text{lm}} (a + a) * X \\ &\Rightarrow_{\text{lm}} a + a * a \end{aligned}$$

Semantic Meaning: $(a + a) * a$
Associates + before *.

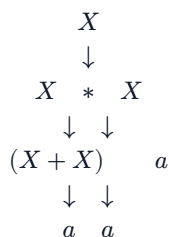
Leftmost Derivation 2 (LMDT-2):
(Addition parsed at root: $X + X$)

$$\begin{aligned} X &\Rightarrow_{\text{lm}} X + X \\ &\Rightarrow_{\text{lm}} a + X \\ &\Rightarrow_{\text{lm}} a + (X * X) \\ &\Rightarrow_{\text{lm}} a + (a * X) \\ &\Rightarrow_{\text{lm}} a + a * a \end{aligned}$$

Semantic Meaning: $a + (a * a)$
Associates * before + (Standard precedence).

3. Construction of Two Distinct Parse Trees [2 Marks]

Parse Tree 1: [(a + a) times a]



Parse Tree 2: [a + (a times a)]

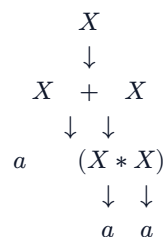


Figure 1.1: Two Distinct Parse Trees for the Same String 'a + a * a'

4. Rightmost Derivations Verification [1 Mark]

For complete rigour, two distinct **Rightmost Derivations (RMDT)** also exist for $w = a + a * a$:

- **RMDT-1:** $X \Rightarrow_{\text{rm}} X * X \Rightarrow_{\text{rm}} X * a \Rightarrow_{\text{rm}} (X + X) * a \Rightarrow_{\text{rm}} (X + a) * a \Rightarrow_{\text{rm}} a + a * a$
- **RMDT-2:** $X \Rightarrow_{\text{rm}} X + X \Rightarrow_{\text{rm}} X + (X * X) \Rightarrow_{\text{rm}} X + (X * a) \Rightarrow_{\text{rm}} X + a * a \Rightarrow_{\text{rm}} a + a * a$

5. Formal Conclusion [1 Mark]

- **Conclusion:** Since there exist two completely different Leftmost Derivations (LMDT) and two structurally distinct Parse Trees for the single terminal string $w = a + a * a$, the grammar:

$$X \rightarrow X + X \mid X * X \mid a$$

is unequivocally **AMBIGUOUS**.

- **Engineering Consequence:** The calculator parser cannot decide whether $2 + 3 * 4$ evaluates to $(2 + 3) * 4 = 20$ or $2 + (3 * 4) = 14$. To fix this in practice, the grammar must be rewritten into an unambiguous grammar with explicit precedence levels ($E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow a$).

Question 2: Conversion of Right-Linear Grammar (RLG) to Left-Linear Grammar (LLG) [8 Marks]

SCENARIO:

A software application uses a finite-state pattern validator represented by the following Right-Linear Grammar (RLG):

$$S \rightarrow aS \mid bA$$

$$A \rightarrow aA \mid b$$

Question: Apply the RLG-to-LLG conversion technique to this grammar. Construct the equivalent Finite Automaton (FA) and obtain the corresponding Left-Linear Grammar (LLG). [8 Marks]

1. Theoretical Principle of RLG-to-LLG Conversion [1 Mark]

- A **Right-Linear Grammar (RLG)** has productions of the form $A \rightarrow wB$ or $A \rightarrow w$ (variables appear on the extreme right).
- A **Left-Linear Grammar (LLG)** has productions of the form $A \rightarrow Bw$ or $A \rightarrow w$ (variables appear on the extreme left).
- **Standard Conversion Algorithm:**
 1. **Step 1:** Convert the RLG into an equivalent Nondeterministic/Deterministic Finite Automaton (M).
 2. **Step 2:** Reverse the Finite Automaton to accept the reversed language L^R , or use the state-transition reversal technique:
 - Reverse all transition arrows in the FA.
 - Swap the initial and final states (the former final state becomes the start state; the former start state becomes the final state).
 3. **Step 3:** Write the regular grammar from the reversed automaton and obtain the equivalent Left-Linear Grammar directly.

2. Step 1 — Construct the Finite Automaton (M) from RLG [2 Marks]

From the given RLG:

- Productions:
 - $S \rightarrow aS \rightarrow$ On state S , reading a loops to S .
 - $S \rightarrow bA \rightarrow$ On state S , reading b transitions to A .
 - $A \rightarrow aA \rightarrow$ On state A , reading a loops to A .
 - $A \rightarrow b \rightarrow$ On state A , reading b reaches an accepting final state (let this be state B or F).
- Introducing explicit final state B :
 - The grammar becomes:

$$S \rightarrow aS \mid bA$$

$$A \rightarrow aA \mid bB$$

$$B \rightarrow \epsilon$$

- **Finite Automaton Components:**

- States: $Q = \{S, A, B\}$
- Start State: S
- Final State: $F = \{B\}$
- Transitions:
 - $\delta(S, a) = S$
 - $\delta(S, b) = A$
 - $\delta(A, a) = A$
 - $\delta(A, b) = B$

3. Step 2 — Construct the Reversed Automaton (M^R) [2 Marks]

Reverse all transitions and swap initial/final states:

- New Start State: B (the former final state)
- New Final State: S (the former start state)
- Reversed Transitions:
 - Original $\delta(S, a) = S \rightarrow$ Reversed: $\delta^{R(S,a)} = S$
 - Original $\delta(S, b) = A \rightarrow$ Reversed: $\delta^{R(A,b)} = S$
 - Original $\delta(A, a) = A \rightarrow$ Reversed: $\delta^{R(A,a)} = A$
 - Original $\delta(A, b) = B \rightarrow$ Reversed: $\delta^{R(B,b)} = A$

4. Step 3 — Construct the Left-Linear Grammar (LLG) [3 Marks]

Using the standard rule: A transition from state P to state Q on input symbol x ($P \xrightarrow{x} Q$) in the forward automaton corresponds to an LLG production:

$$Q \rightarrow Px$$

If P is the start state, then $Q \rightarrow x$ is also added (or $S \rightarrow \varepsilon$ if start state is final).

Mapping the forward automaton transitions to LLG productions:

1. Forward: $S \xrightarrow{a} S \rightarrow$ LLG: $S \rightarrow Sa$
2. Forward: $S \xrightarrow{b} A \rightarrow$ LLG: $A \rightarrow Sb$
3. Forward: $A \xrightarrow{a} A \rightarrow$ LLG: $A \rightarrow Aa$
4. Forward: $A \xrightarrow{b} B \rightarrow$ LLG: $B \rightarrow Ab$
5. Initial condition: S is start state, so $S \rightarrow aS$ gives base production $S \rightarrow a$.

Equivalently, writing the productions rooted at final state B (or setting B as new start symbol): Let B be the start symbol of the LLG:

$$\begin{aligned} B &\rightarrow Ab \\ A &\rightarrow Aa \mid Sb \\ S &\rightarrow Sa \mid a \mid b \end{aligned}$$

Verification with Language:

- The language is $L = a^*ba^*b$.
- Generates strings: $bb, abb, bab, aabaab$, etc.
- LLG derivation for abb :

$$B \Rightarrow Ab \Rightarrow Sbb \Rightarrow abb$$

- Both grammars generate the exact same regular language $L(G) = a^*ba^*b$.

Question 3: Practice RLG-to-LLG Conversions [8 Marks]

SCENARIO:

Problem 3A (Data Validation Module):

$$S \rightarrow aS \mid bA$$

$$A \rightarrow aB \mid bA$$

$$B \rightarrow a \mid bS$$

Convert this Right-Linear Grammar to its equivalent Left-Linear Grammar.

Solution for Problem 3A: Data Validation Module RLG to LLG [4 Marks]**1. Forward Transitions in FA:**

- $S \xrightarrow{a} S$
- $S \xrightarrow{b} A$
- $A \xrightarrow{a} B$
- $A \xrightarrow{b} A$
- $B \xrightarrow{b} S$
- $B \xrightarrow{a} F$ (where F is the final accepting state, from $B \rightarrow a$)

2. Conversion to LLG: Using $Q \rightarrow Px$ for every forward transition $P \xrightarrow{x} Q$:

- From $S \xrightarrow{a} S \rightarrow S \rightarrow Sa \mid a$
- From $S \xrightarrow{b} A \rightarrow A \rightarrow Sb \mid b$
- From $A \xrightarrow{a} B \rightarrow B \rightarrow Ab$
- From $A \xrightarrow{b} A \rightarrow A \rightarrow Ab$
- From $B \xrightarrow{b} S \rightarrow S \rightarrow Bb$
- From $B \xrightarrow{a} F \rightarrow F \rightarrow Ba$

Resulting Left-Linear Grammar (with F as start symbol):

$$\begin{aligned} F &\rightarrow Ba \\ B &\rightarrow Ab \\ A &\rightarrow Ab \mid Sb \mid b \\ S &\rightarrow Sa \mid Bb \mid a \end{aligned}$$

SCENARIO:**Problem 3B (Traffic Signal Sequence):**

$$S \rightarrow aA \mid bS$$

$$A \rightarrow aB \mid b$$

$$B \rightarrow aS \mid bA$$

Convert this Right-Linear Grammar to equivalent Finite Automaton and Left-Linear Grammar.

Solution for Problem 3B: Traffic Signal Sequence RLG to LLG [4 Marks]**1. Forward Transitions:**

- $S \xrightarrow{a} A, S \xrightarrow{b} S$
- $A \xrightarrow{a} B, A \xrightarrow{b} F$ (final state from $A \rightarrow b$)
- $B \xrightarrow{a} S, B \xrightarrow{b} A$

2. Resulting Left-Linear Grammar (with F as start symbol):

$$\begin{aligned} F &\rightarrow Ab \\ A &\rightarrow Sa \mid Bb \mid a \\ B &\rightarrow Aa \\ S &\rightarrow Sb \mid Ba \mid b \end{aligned}$$

Question 4: Case Study Derivations (LMDT & RMDT) [8 Marks Each]

SCENARIO:

Case Study 4A: Online Railway Ticket Booking System A railway ticket booking system uses the following CFG to generate booking commands:

$$S \rightarrow AB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

Question: Derive the command string $w = aaabb$ using: a) Leftmost Derivation (LMDT) [4 Marks] b) Rightmost Derivation (RMDT) [4 Marks]

Solution 4A: Railway Ticket Booking System ($w = aaabb$)

a) Leftmost Derivation (LMDT) [4 Marks]

At each step, replace the leftmost non-terminal:

$$\begin{aligned} S &\Rightarrow_{\text{lm}} AB && [\text{Apply } S \rightarrow AB] \\ &\Rightarrow_{\text{lm}} aAB && [\text{Apply } A \rightarrow aA] \\ &\Rightarrow_{\text{lm}} aaAB && [\text{Apply } A \rightarrow aA] \\ &\Rightarrow_{\text{lm}} aaaB && [\text{Apply } A \rightarrow a] \\ &\Rightarrow_{\text{lm}} aaabB && [\text{Apply } B \rightarrow bB] \\ &\Rightarrow_{\text{lm}} aaabb && [\text{Apply } B \rightarrow b] \end{aligned}$$

Terminal String Produced: $aaabb$

b) Rightmost Derivation (RMDT) [4 Marks]

At each step, replace the rightmost non-terminal:

$$\begin{aligned} S &\Rightarrow_{\text{rm}} AB && [\text{Apply } S \rightarrow AB] \\ &\Rightarrow_{\text{rm}} AbB && [\text{Apply } B \rightarrow bB] \\ &\Rightarrow_{\text{rm}} Abb && [\text{Apply } B \rightarrow b] \\ &\Rightarrow_{\text{rm}} aAbb && [\text{Apply } A \rightarrow aA] \\ &\Rightarrow_{\text{rm}} aaAbb && [\text{Apply } A \rightarrow aA] \\ &\Rightarrow_{\text{rm}} aaabb && [\text{Apply } A \rightarrow a] \end{aligned}$$

Terminal String Produced: $aaabb$

SCENARIO:

Case Study 4B: Login Authentication Module The grammar is:

$$S \rightarrow AB$$

$$A \rightarrow 0A \mid 0$$

$$B \rightarrow 1B \mid 1$$

Question: Derive the authentication sequence $w = 00011$ using LMDT and RMDT. [8 Marks]

Solution 4B: Login Authentication Module ($w = 00011$)**a) Leftmost Derivation (LMDT) [4 Marks]**

$$\begin{aligned}
 S &\Rightarrow_{\text{lm}} AB & [S \rightarrow AB] \\
 &\Rightarrow_{\text{lm}} 0AB & [A \rightarrow 0A] \\
 &\Rightarrow_{\text{lm}} 00AB & [A \rightarrow 0A] \\
 &\Rightarrow_{\text{lm}} 000B & [A \rightarrow 0] \\
 &\Rightarrow_{\text{lm}} 0001B & [B \rightarrow 1B] \\
 &\Rightarrow_{\text{lm}} 00011 & [B \rightarrow 1]
 \end{aligned}$$

b) Rightmost Derivation (RMDT) [4 Marks]

$$\begin{aligned}
 S &\Rightarrow_{\text{rm}} AB & [S \rightarrow AB] \\
 &\Rightarrow_{\text{rm}} A1B & [B \rightarrow 1B] \\
 &\Rightarrow_{\text{rm}} A11 & [B \rightarrow 1] \\
 &\Rightarrow_{\text{rm}} 0A11 & [A \rightarrow 0A] \\
 &\Rightarrow_{\text{rm}} 00A11 & [A \rightarrow 0A] \\
 &\Rightarrow_{\text{rm}} 00011 & [A \rightarrow 0]
 \end{aligned}$$

SCENARIO:

Case Study 4C: Student Record System The grammar is:

$$\begin{aligned}
 S &\rightarrow AB \\
 A &\rightarrow pA \mid p \\
 B &\rightarrow qB \mid q
 \end{aligned}$$

Question: Derive the record pattern $w = ppqqq$ using LMDT and RMDT. [8 Marks]

Solution 4C: Student Record System ($w = ppqqq$)**a) Leftmost Derivation (LMDT) [4 Marks]**

$$\begin{aligned}
 S &\Rightarrow_{\text{lm}} AB & [S \rightarrow AB] \\
 &\Rightarrow_{\text{lm}} pAB & [A \rightarrow pA] \\
 &\Rightarrow_{\text{lm}} ppB & [A \rightarrow p] \\
 &\Rightarrow_{\text{lm}} ppqB & [B \rightarrow qB] \\
 &\Rightarrow_{\text{lm}} ppqqB & [B \rightarrow qB] \\
 &\Rightarrow_{\text{lm}} ppqqq & [B \rightarrow q]
 \end{aligned}$$

b) Rightmost Derivation (RMDT) [4 Marks]

$$\begin{aligned}
 S &\Rightarrow_{\text{rm}} AB & [S \rightarrow AB] \\
 &\Rightarrow_{\text{rm}} AqB & [B \rightarrow qB] \\
 &\Rightarrow_{\text{rm}} AqqB & [B \rightarrow qB] \\
 &\Rightarrow_{\text{rm}} Aqqq & [B \rightarrow q] \\
 &\Rightarrow_{\text{rm}} pAqqq & [A \rightarrow pA] \\
 &\Rightarrow_{\text{rm}} ppqqq & [A \rightarrow p]
 \end{aligned}$$

SCENARIO:

Case Study 4D: ATM Transaction Sequence The grammar is:

$$\begin{aligned}
 S &\rightarrow AB \\
 A &\rightarrow 1A \mid 1 \\
 B &\rightarrow 2B \mid 2
 \end{aligned}$$

Question: Derive the transaction sequence $w = 111222$ using LMDT and RMDT. [8 Marks]

Solution 4D: ATM Transaction Sequence ($w = 111222$)**a) Leftmost Derivation (LMDT) [4 Marks]**

$$\begin{aligned}
S &\Rightarrow_{\text{lm}} AB & [S \rightarrow AB] \\
&\Rightarrow_{\text{lm}} 1AB & [A \rightarrow 1A] \\
&\Rightarrow_{\text{lm}} 11AB & [A \rightarrow 1A] \\
&\Rightarrow_{\text{lm}} 111B & [A \rightarrow 1] \\
&\Rightarrow_{\text{lm}} 1112B & [B \rightarrow 2B] \\
&\Rightarrow_{\text{lm}} 11122B & [B \rightarrow 2B] \\
&\Rightarrow_{\text{lm}} 111222 & [B \rightarrow 2]
\end{aligned}$$

b) Rightmost Derivation (RMDT) [4 Marks]

$$\begin{aligned}
S &\Rightarrow_{\text{rm}} AB & [S \rightarrow AB] \\
&\Rightarrow_{\text{rm}} A2B & [B \rightarrow 2B] \\
&\Rightarrow_{\text{rm}} A22B & [B \rightarrow 2B] \\
&\Rightarrow_{\text{rm}} A222 & [B \rightarrow 2] \\
&\Rightarrow_{\text{rm}} 1A222 & [A \rightarrow 1A] \\
&\Rightarrow_{\text{rm}} 11A222 & [A \rightarrow 1A] \\
&\Rightarrow_{\text{rm}} 111222 & [A \rightarrow 1]
\end{aligned}$$

SCENARIO:**Problem 4E: Additional Handwritten Derivation Examples from Slides (Page 6)**

1. $w = aabbabba$ for grammar:

$$S \rightarrow aB \mid bA, \quad A \rightarrow a \mid aS \mid bAA, \quad B \rightarrow b \mid bS \mid aBB$$

2. $w = abb$ for grammar:

$$S \rightarrow AB \mid \varepsilon, \quad A \rightarrow aB, \quad B \rightarrow Sb$$

3. $w = 00101$ for grammar:

$$S \rightarrow A \mid B, \quad A \rightarrow 0A \mid \varepsilon, \quad B \rightarrow 0B \mid 1B \mid \varepsilon$$

Solution 4E: Additional Grammar Derivations [8 Marks]

1. Derivation for $w = abb$ from $S \rightarrow AB \mid \varepsilon, A \rightarrow aB, B \rightarrow Sb$:

• **Leftmost Derivation (LMDT):**

$$S \Rightarrow_{\text{lm}} AB \Rightarrow_{\text{lm}} aBB \Rightarrow_{\text{lm}} a(Sb)B \Rightarrow_{\text{lm}} a(\varepsilon b)B \Rightarrow_{\text{lm}} ab(Sb) \Rightarrow_{\text{lm}} ab(\varepsilon b) = abb$$

• **Rightmost Derivation (RMDT):**

$$S \Rightarrow_{\text{rm}} AB \Rightarrow_{\text{rm}} A(Sb) \Rightarrow_{\text{rm}} A(Sb) \Rightarrow_{\text{rm}} A(\varepsilon b) \Rightarrow_{\text{rm}} (aB)b \Rightarrow_{\text{rm}} (a(Sb))b \Rightarrow_{\text{rm}} (a(\varepsilon b))b = abb$$

2. Derivation for $w = 00101$ from $S \rightarrow A \mid B, A \rightarrow 0A \mid \varepsilon, B \rightarrow 0B \mid 1B \mid \varepsilon$:

• **Leftmost Derivation (LMDT):**

$$S \Rightarrow_{\text{lm}} B \Rightarrow_{\text{lm}} 0B \Rightarrow_{\text{lm}} 00B \Rightarrow_{\text{lm}} 001B \Rightarrow_{\text{lm}} 0010B \Rightarrow_{\text{lm}} 00101B \Rightarrow_{\text{lm}} 00101(\varepsilon) = 00101$$

• **Rightmost Derivation (RMDT):** Since each production in B has at most one non-terminal appearing at the extreme right, the Rightmost Derivation follows the exact same sequence of production choices as the Leftmost Derivation.

UNIT IV: PUSHDOWN AUTOMATA (PDA) CASE STUDIES

Continuous Concurrent Evaluation (CCE2) — Comprehensive Worked Solutions

Course Pattern: TE IT - 2024 Pattern (SPPU)
Document Type: CCE2 Model Answer Solutions

Academic Term: Semester V (2026)
Course Code: PCC-303-ITT

NOTE:

This document provides exhaustive, 8-mark worked solutions for all Pushdown Automata (PDA) real-world case studies in the CCE2 question bank. Each case study contains: (1) Language identification and requirement analysis, (2) Stack operational intuition, (3) Formal 7-tuple definition, (4) Transition functions, (5) State transition diagram, (6) Complete Instantaneous Description (ID) trace, and (7) Acceptance criteria justification.

Case Study 1: Restaurant Dish Order Verification Using PDA [8 Marks]

SCENARIO:

A restaurant serves Starter dishes (a) first, followed by an equal number of Main Course dishes (b). For every Starter dish served, exactly one Main Course dish must be served. The valid order sequence is represented by the language:

$$L = \{a^n b^n \mid n \geq 1\}$$

Question: Design a Pushdown Automaton (PDA) that recognizes the above language. Show the push and pop operations and trace the PDA for the input string $w = aabb$ using Instantaneous Description (ID).

1. Language Identification & Requirement Analysis [1 Mark]

- **Input Alphabet:** $\Sigma = \{a, b\}$, where a = Starter dish, b = Main Course dish.
- **Formal Language:** $L = \{a^n b^n \mid n \geq 1\} = \{ab, aabb, aaabbb, a^4 b^4, \dots\}$.
- **Requirement:** All Starter dishes (a) must be served strictly before any Main Course dish (b). For each a served, there must be exactly one corresponding b . Strings with mismatched counts (e.g., $aab, abb, aaabb$) or interleaved sequences (e.g., $abab$) must be rejected.

2. Stack-Based Operational Intuition [1 Mark]

- A finite automaton cannot recognize $L = \{a^n b^n \mid n \geq 1\}$ because it lacks unbounded counting memory.
- A **Pushdown Automaton (PDA)** utilizes a Last-In, First-Out (LIFO) stack:
 1. **Phase 1 (Reading Starters):** In state q_0 , for every Starter symbol a read, push a stack symbol A onto the stack.
 2. **Phase 2 (Switching to Main Courses):** On reading the first b , transition from state q_0 to state q_1 and pop one A from the stack.
 3. **Phase 3 (Reading Subsequent Main Courses):** In state q_1 , for every subsequent b , pop one A from the stack.
 4. **Phase 4 (Acceptance):** When all input is consumed and the top of the stack is the start symbol Z_0 , transition on ε to final state q_f .

3. Formal 7-Tuple Definition of PDA [1 Mark]

The PDA is formally defined as a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

- $Q = \{q_0, q_1, q_f\}$ — Finite set of states:
 - q_0 : Initial state / Reading Starter dishes (a)
 - q_1 : Reading Main Course dishes (b)
 - q_f : Final accepting state
- $\Sigma = \{a, b\}$ — Input alphabet
- $\Gamma = \{A, Z_0\}$ — Stack alphabet (Z_0 : initial stack symbol, A : marker for dish a)
- $q_0 \in Q$ — Initial start state
- $Z_0 \in \Gamma$ — Initial stack start symbol
- $F = \{q_f\}$ — Set of accepting states (Acceptance by Final State)

4. Transition Functions (δ) [2 Marks]

Transition Equation	Stack Action	Operational Meaning
$\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\}$	Push A	Read first Starter a ; push A over Z_0
$\delta(q_0, a, A) = \{(q_0, AA)\}$	Push A	Read subsequent Starter a ; push A over A
$\delta(q_0, b, A) = \{(q_1, \varepsilon)\}$	Pop A	Read first Main Course b ; change state to q_1 and pop A
$\delta(q_1, b, A) = \{(q_1, \varepsilon)\}$	Pop A	Read subsequent Main Course b ; pop one A
$\delta(q_1, \varepsilon, Z_0) = \{(q_f, Z_0)\}$	No change	Input empty & stack has Z_0 ; move to final state q_f

5. State Transition Diagram [1 Mark]

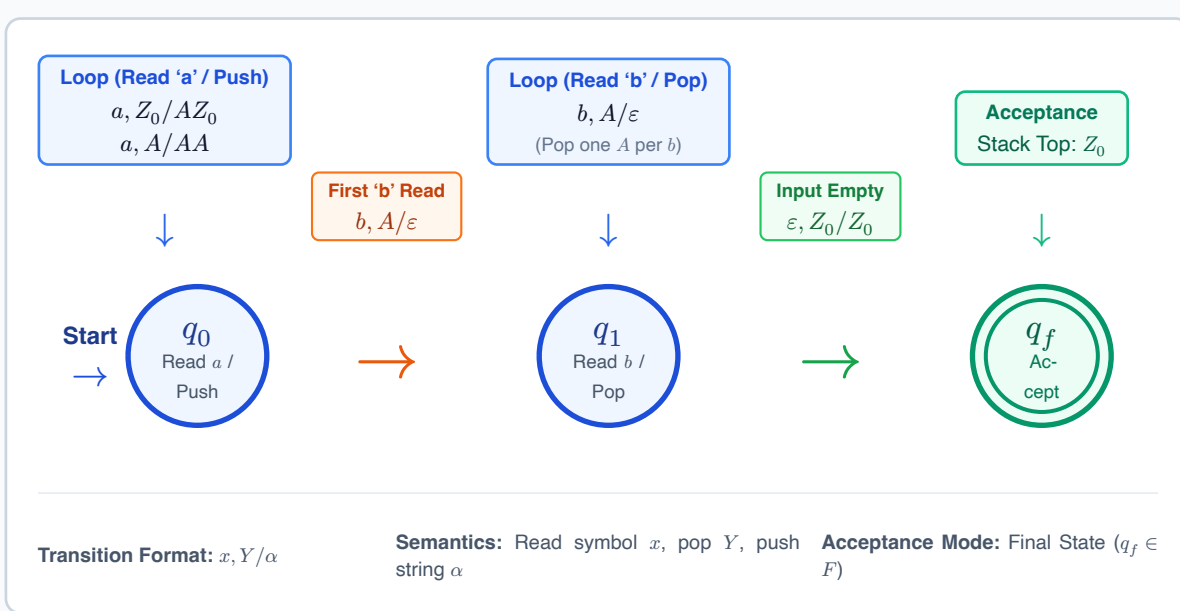


Figure 1.1: State Transition Diagram of PDA for Restaurant Dish Verification

6. Instantaneous Description (ID) Trace for Input $w = aabb$ [1 Mark]

An Instantaneous Description of a PDA is written as (q, w, α) , where q is the current state, w is the unconsumed input, and α is the stack string (leftmost symbol is top-of-stack).

$(q_0, aabb, Z_0) \mid \rightarrow (q_0, abb, AZ_0)$ [Read 'a', Push A]
 $\mid \rightarrow (q_0, bb, AAZ_0)$ [Read 'a', Push A]
 $\mid \rightarrow (q_1, b, AZ_0)$ [Read 'b', Change to q_1 , Pop A]
 $\mid \rightarrow (q_1, \epsilon, Z_0)$ [Read 'b', Pop A]
 $\mid \rightarrow (q_f, \epsilon, Z_0)$ [Input empty, Transition to final state q_f]

Step	Input Left	State	Stack (Top $\rightarrow$)	Action Taken
0	$aabb$	q_0	Z_0	Initial configuration
1	abb	q_0	AZ_0	Read a ; push A
2	bb	q_0	AAZ_0	Read a ; push A
3	b	q_1	AZ_0	Read b ; transition to q_1 , pop A
4	ϵ	q_1	Z_0	Read b ; pop A
5	ϵ	q_f	Z_0	Epsilon move; enter final state q_f

7. Acceptance Criteria Justification [1 Mark]

- **String $aabb$ is ACCEPTED.**
- **Justification:** All input symbols have been completely consumed ($w = \varepsilon$), and the PDA enters the designated final state $q_f \in F$. Thus, $(q_0, aabb, Z_0) \mid \vdash^* (q_f, \varepsilon, Z_0)$, confirming $aabb \in L$.

Case Study 2: Library Book Arrangement System [8 Marks]

SCENARIO:

A library arranges fiction books (a) first, followed by an equal number of reference books (b). For every fiction book placed, exactly one reference book must be placed. The valid sequence is represented by:

$$L = \{a^n b^n \mid n \geq 1\}$$

Question: Apply the concept of Pushdown Automata (PDA) to this case study and design a PDA that recognizes the above language. Show the push and pop operations and trace the PDA for the input string $w = aaabbb$ using Instantaneous Description (ID).

1. Problem Formulation & Formal Definition [2 Marks]

- Let a represent a fiction book placed on the shelf.
- Let b represent a reference book placed on the shelf.
- Required language: $L = \{a^n b^n \mid n \geq 1\} = \{ab, aabb, aaabbb, \dots\}$.
- The PDA is defined as: $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
 - $Q = \{q_0, q_1, q_f\}$
 - $\Sigma = \{a, b\}, \Gamma = \{A, Z_0\}$
 - Start state: q_0 , Initial stack symbol: Z_0 , Final state: $F = \{q_f\}$

2. Transition Rules [2 Marks]

1. $\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\} \rightarrow$ Read first fiction book a ; push A onto stack.
2. $\delta(q_0, a, A) = \{(q_0, AA)\} \rightarrow$ Read subsequent fiction book a ; push A onto stack.
3. $\delta(q_0, b, A) = \{(q_1, \varepsilon)\} \rightarrow$ Read first reference book b ; change to state q_1 and pop A .
4. $\delta(q_1, b, A) = \{(q_1, \varepsilon)\} \rightarrow$ Read subsequent reference book b ; pop A .
5. $\delta(q_1, \varepsilon, Z_0) = \{(q_f, Z_0)\} \rightarrow$ All books verified; transition to accepting state q_f .

3. Instantaneous Description (ID) Trace for $w = aaabbb$ [3 Marks]

$(q_0, aaabbb, Z_0) \mid \rightarrow (q_0, aabbb, AZ_0)$ [Read 'a', Push A]
 $\mid \rightarrow (q_0, abbb, AAZ_0)$ [Read 'a', Push A]
 $\mid \rightarrow (q_0, bbb, AAAZ_0)$ [Read 'a', Push A]
 $\mid \rightarrow (q_1, bb, AAZ_0)$ [Read 'b', Change to q_1 , Pop A]
 $\mid \rightarrow (q_1, b, AZ_0)$ [Read 'b', Pop A]
 $\mid \rightarrow (q_1, \varepsilon, Z_0)$ [Read 'b', Pop A]
 $\mid \rightarrow (q_f, \varepsilon, Z_0)$ [Input empty, Transition to final state q_f]

Step	Input Left	State	Stack	Action
0	aaabbb	q_0	Z_0	Initial setup
1	aabbb	q_0	AZ_0	Read a; push A
2	abbb	q_0	AAZ_0	Read a; push A
3	bbb	q_0	$AAAZ_0$	Read a; push A (3 fiction books on stack)
4	bb	q_1	AAZ_0	Read b; change state to q_1 , pop A
5	b	q_1	AZ_0	Read b; pop A
6	ε	q_1	Z_0	Read b; pop A (All A's matched)
7	ε	q_f	Z_0	Input empty; move to final state q_f

5. Conclusion & Acceptance Decision [1 Mark]

- **Final Result:** Since the input string is fully read and the PDA is in final state q_f with stack containing Z_0 , the string $aaabbb$ is **ACCEPTED** ($aaabbb \in L$).

Case Study 3: Service Center Registration & Service Verification [8 Marks]

SCENARIO:

A service center first registers customers (a) and then serves an equal number of customers (b). Every registered customer must correspond to exactly one completed service. Valid sequence: $L = \{a^n b^n \mid n \geq 1\}$. **Question:** Design a PDA for this language. Clearly show how the PDA pushes a symbol for every registration and pops one symbol for every service. Trace the PDA for $aaabbb$ using Instantaneous Descriptions.

1. Conceptual Operation & Formal Model [2 Marks]

- For each registration event (a), the PDA increments its count by pushing a token A onto the stack.
- For each completed service event (b), the PDA decrements its count by popping one token A .
- Formal 7-Tuple: $M = (\{q_0, q_1, q_f\}, \{a, b\}, \{A, Z_0\}, \delta, q_0, Z_0, \{q_f\})$.

2. Push & Pop Transition Specifications [2 Marks]

- **Registration Phase (Pushing):**
 - $\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\}$ — Register first customer; push A over Z_0 .
 - $\delta(q_0, a, A) = \{(q_0, AA)\}$ — Register next customer; push A over existing A .
- **Service Phase (Popping):**
 - $\delta(q_0, b, A) = \{(q_1, \varepsilon)\}$ — First service served; transition to service phase q_1 and pop A .
 - $\delta(q_1, b, A) = \{(q_1, \varepsilon)\}$ — Next service served; pop one A from stack.
- **Verification Phase (Final Acceptance):**
 - $\delta(q_1, \varepsilon, Z_0) = \{(q_f, Z_0)\}$ — All registered customers matched with services; accept in q_f .

3. Instantaneous Description Trace for $w = aaabbb$ [3 Marks]

$(q_0, aaabbb, Z_0)$		$-(q_0, aaabbb, AZ_0)$	[Customer 1 registered: Push A]
		$-(q_0, abbb, AAZ_0)$	[Customer 2 registered: Push A]
		$-(q_0, bbb, AAAZ_0)$	[Customer 3 registered: Push A]
		$-(q_1, bb, AAZ_0)$	[Customer 1 served: Pop A, enter service phase q_1]
		$-(q_1, b, AZ_0)$	[Customer 2 served: Pop A]
		$-(q_1, \varepsilon, Z_0)$	[Customer 3 served: Pop A]
		$-(q_f, \varepsilon, Z_0)$	[All services complete: Accept in q_f]

5. Result & Error Rejection [1 Mark]

- **Acceptance:** The string $aaabbb$ has equal counts of registrations and services and is successfully accepted by reaching state q_f .
- **Rejection of Violations:**
 - If extra registrations occur (e.g., $aaabb$), the stack retains unmatched A 's when input ends, and the PDA cannot reach q_f (Rejection).
 - If extra services occur (e.g., $aabbb$), the stack encounters Z_0 while input still has b , halting in a non-final configuration (Rejection).